

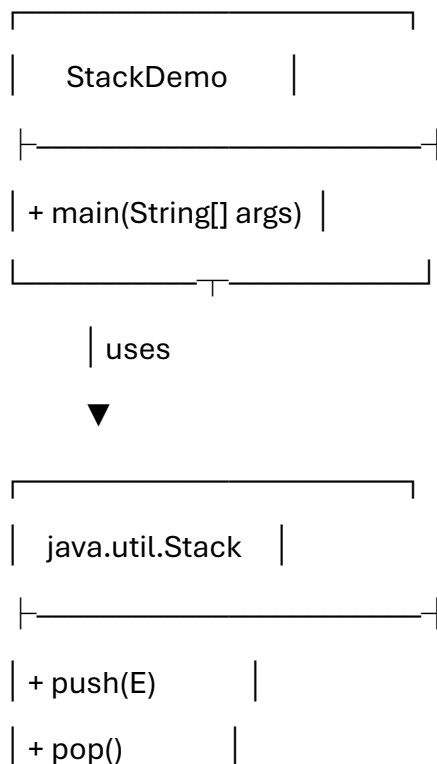
Aim

To write a Java program to demonstrate the important methods of the **Stack class**, such as `push()`, `pop()`, `peek()`, `empty()`, and `search()`.

Algorithm

1. Start the program.
2. Import the `java.util.Stack` class.
3. Create a Stack object.
4. Use `push()` to add elements to the stack.
5. Display the stack.
6. Use `peek()` to view the top element without removing it.
7. Use `search()` to find the position of an element.
8. Use `pop()` to remove the top element.
9. Use `empty()` to check whether the stack is empty.
10. Display the results.
11. Stop the program.

UML Diagram



```

| + peek()      |
| + empty()     |
| + search(Object) |
└────────────────┘

```

Program:

```
package Observations;
```

```
import java.util.Stack;
```

```
public class StackClass{
```

```
    public static void main(String[] args) {
```

```
        // Creating a Stack
```

```
        Stack<String> stack = new Stack<>();
```

```
        // push()
```

```
        stack.push("Apple");
```

```
        stack.push("Banana");
```

```
        stack.push("Mango");
```

```
        stack.push("Orange");
```

```
        System.out.println("Stack after push(): " + stack);
```

```
        // peek()
```

```
        System.out.println("Top element using peek(): " + stack.peek());
```

```

// search()

System.out.println("Position of Banana: " + stack.search("Banana"));

// pop()

System.out.println("Removed element using pop(): " + stack.pop());

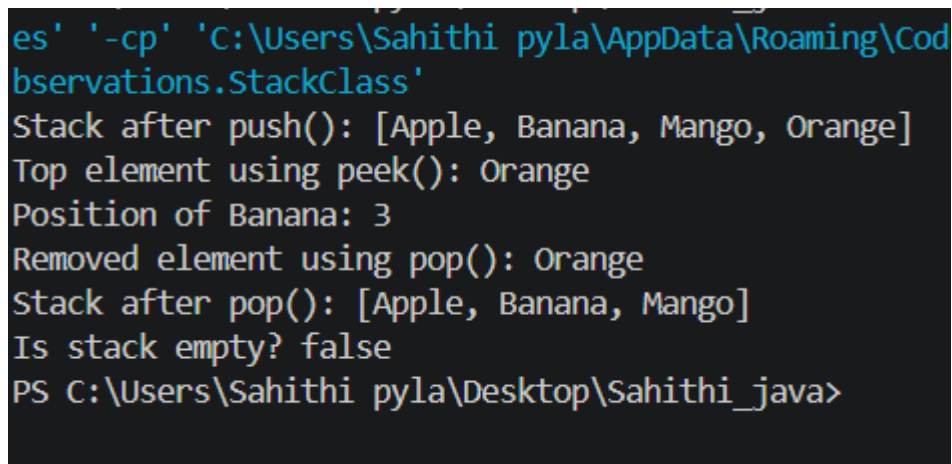
System.out.println("Stack after pop(): " + stack);

// empty()

System.out.println("Is stack empty? " + stack.empty());
}
}

```

Output:



```

es' '-cp' 'C:\Users\Sahithi pyla\AppData\Roaming\Cod
bservations.StackClass'
Stack after push(): [Apple, Banana, Mango, Orange]
Top element using peek(): Orange
Position of Banana: 3
Removed element using pop(): Orange
Stack after pop(): [Apple, Banana, Mango]
Is stack empty? false
PS C:\Users\Sahithi pyla\Desktop\Sahithi_java>

```

Result:

Thus, the Java program successfully demonstrates the Stack class methods push(), pop(), peek(), empty(), and search() using the LIFO principle.